Control Structures – Selection

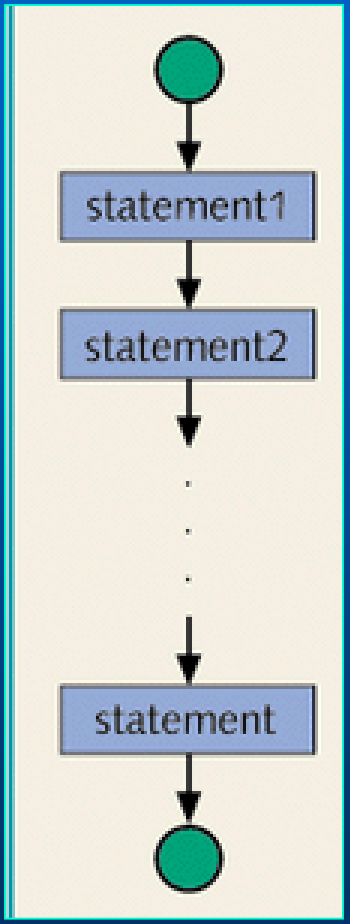
Chapter 4

Chapter Topics

- Control Structures
- Relational Operators
- Logical (Boolean) Operators
- Logical Expressions
- Selection `if ( )` and `if ( ) … else`
- `switch` Structures
- The `assert` Function

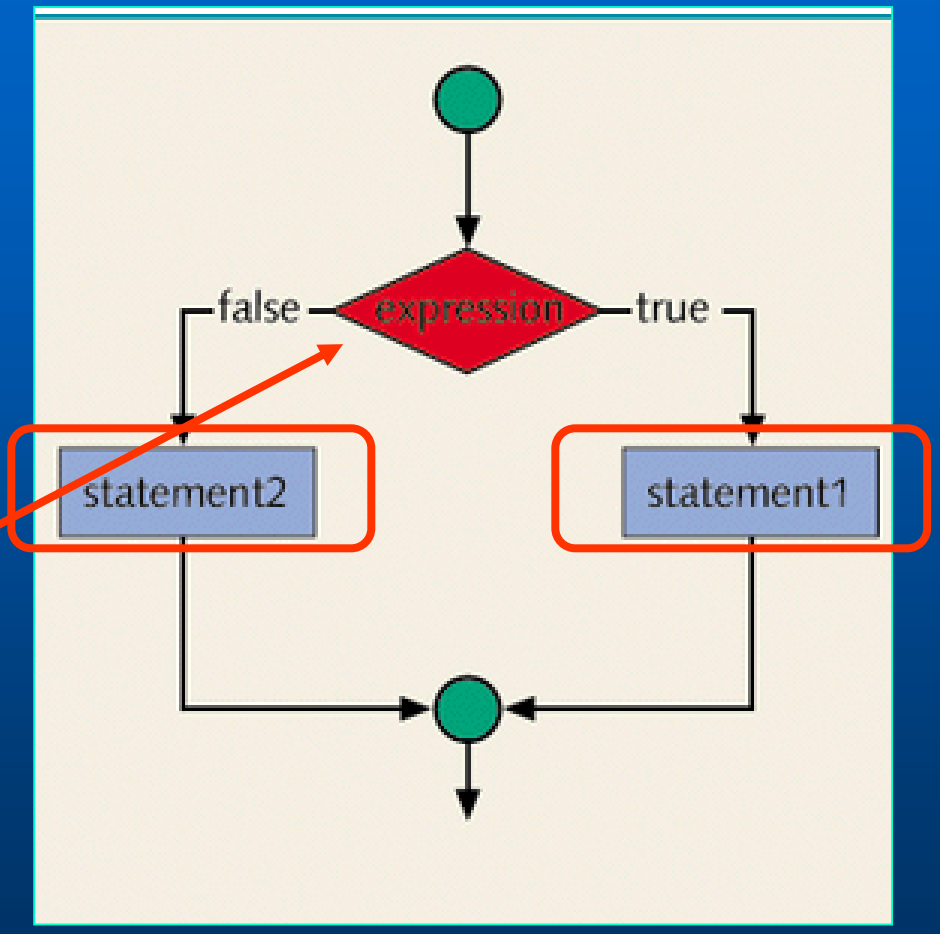
Control Structures

- **Statements can be executed in <u>sequence</u>**
- **One right after the other**
- **No deviation from the specified sequence**



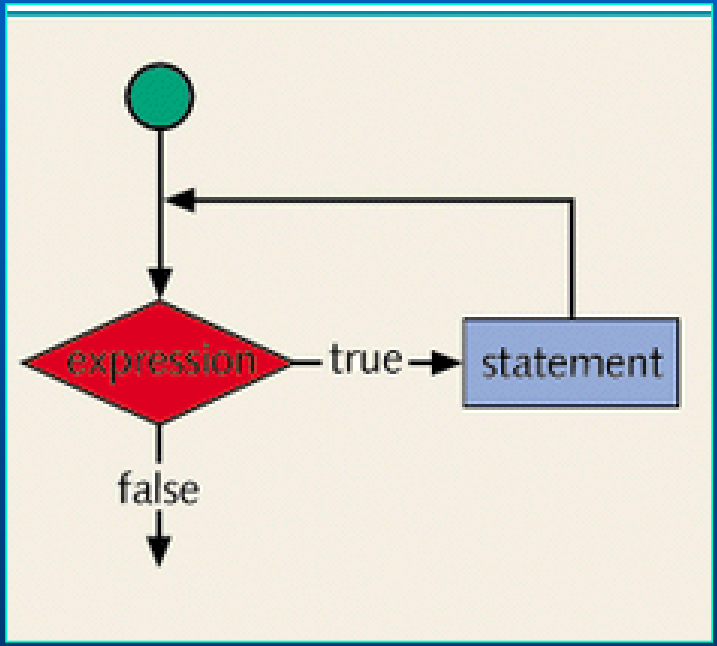
Control Structures

- A selection structure can be used
- Which statement is executed is selected by whether the expression is true or false



Control Structures

- **Statements can be <u>repeated</u>**
- **The number of repetitions depends on when the expression turns false**



Relational Operators

- **The expressions which determine**
  - **Selection and**
  - **Repetition**

  are usually comparisons
- **Comparisons are done with relational operators**

Operator	Description
==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Beware of mistaking the assignment = for the equality ==

Relational Operators

Examples:

Expression	Meaning	Value
`8 < 15`	**8 is less than 15**	`true`
`6 != 6`	**6 is not equal to 6**	`false`
`2.5 > 5.8`	**2.5 is greater than 5.8**	`false`
`5.9 <= 7.5`	**5.9 is less than or equal to 7.5**	`true`

Relational Operators

Given

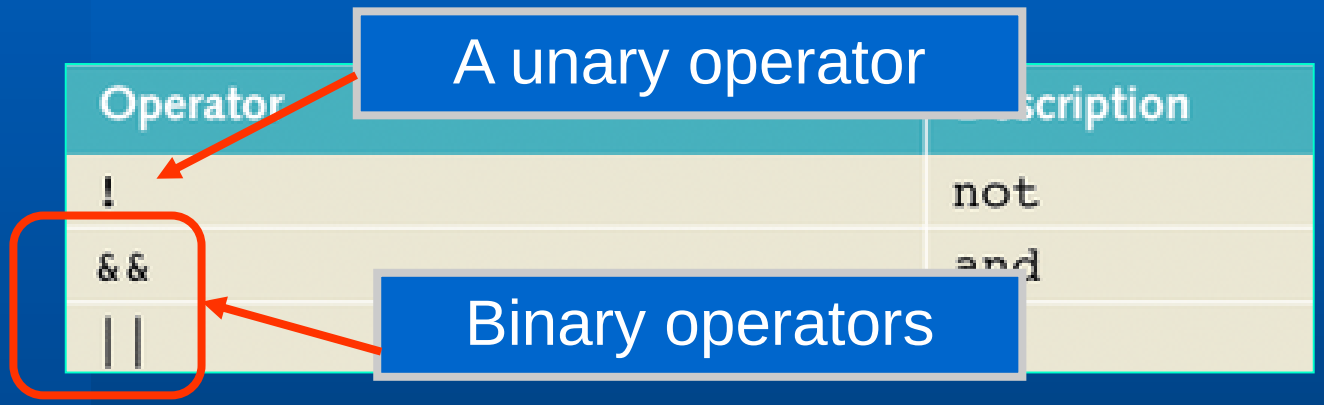
```
string str1 = "Hello";  string str2 = "Hi";
string str3 = "Air";   string str4 = "Bill";
string str5 = "Big";
```

Expression	Value
str1 < str2	true
str1 > "Hen"	false
str3 < "An"	true
str1 == "hello "	false
str3 <= str4	true

ine the values of omparisons using variables

Logical (Boolean) Operators

- **Logical or Boolean operators enable you to combine logical expressions**



- **Operands must be logical values**
- **The results are logical values (true or false)**

Logical (Boolean) Operators

- The && operator (logical and)
  - If both operands are true, the result is true
  - If either or both operands is false, the comparison is false
- The || operator (logical or)
  - If either or both of the operands are true, the comparison is true
  - The comparison is false only if both operands are false
- The ! operator (logical not)
  - The not operator reverses the logical value of the one operand

Logical Expressions

- **We must know the order in which to apply the operators**

```
12 > 7 || 9 * 5 >= 6 && 5 < 9
```

Order of Precedence

Highest

Lowest

Operators	
!, +, -	(unary operators)
*, /, %	
+, -	
<, <=, >=, >	
==, !=	
&&	
||	
=	(assignment operator)

View Sample Program

Short Circuit Evaluation

- Consider
  `(x != 0) && (1.0 / x < 0.25)`
- If the first condition is false, the program could crash when it tried to divide by zero
  - but if the first condition is false, the whole expression is false
  - no need to go on
- When C++ evaluates an expression, realizes that fact and does not even make the second comparison
- Called "short circuit" evaluation

- Unless the || operator is overloaded, the second expression will not be evaluated. This is called "short-circuit evaluation."

- In the case of logical AND (&&) and logical OR (||), the second expression will not be evaluated if the first expression is sufficient to determine the value of the entire expression.

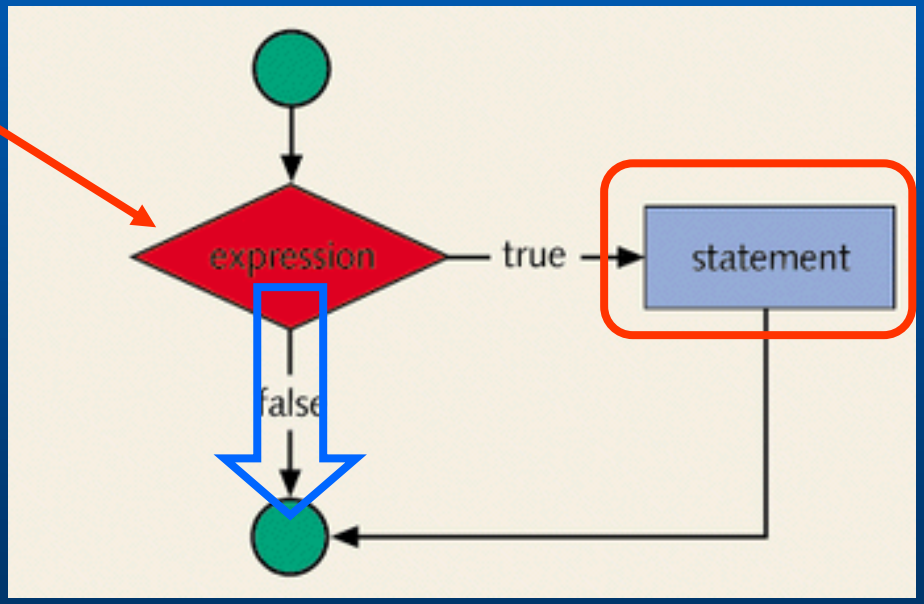
- In the case you described above:

- if(A == 1 || B == 2) {...}
- ...the second expression will not be evaluated because

- TRUE || ANYTHING, always evaluates to TRUE.
- FALSE && ANYTHING, always evaluates to FALSE, so that condition will also cause a short-circuit evaluation.

Selection `if (...)`

- C++ has two versions of if statements
- In this version, the condition is checked
  - If the expression is true, the statement is executed
  - If it is false, nothing happens



Selection `if (...)`

- **Syntax**

```
if ( logicalExpression )
        statement;
```

- **Example**

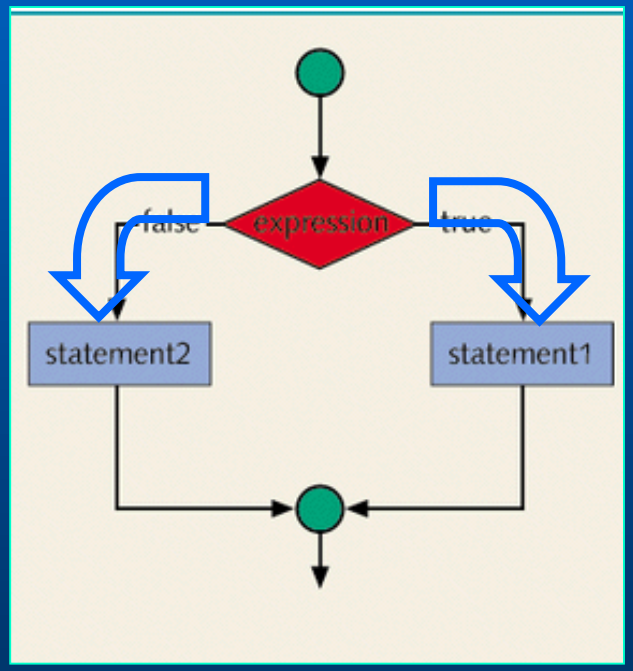
```
if (x < 5 )
     cout << "low value for x";
```

Note parentheses around the condition

Note there is no "then" as part of the syntax

Selection `if ( ) … else …`

- Also possible to make <u>two way</u> selection
- If the expression is true, statement1 is executed
- Otherwise statement2 is executed



Selection `if ( ) … else …`

- **Syntax**

```
if (condition)
    statement1;
else
   statement2;
```

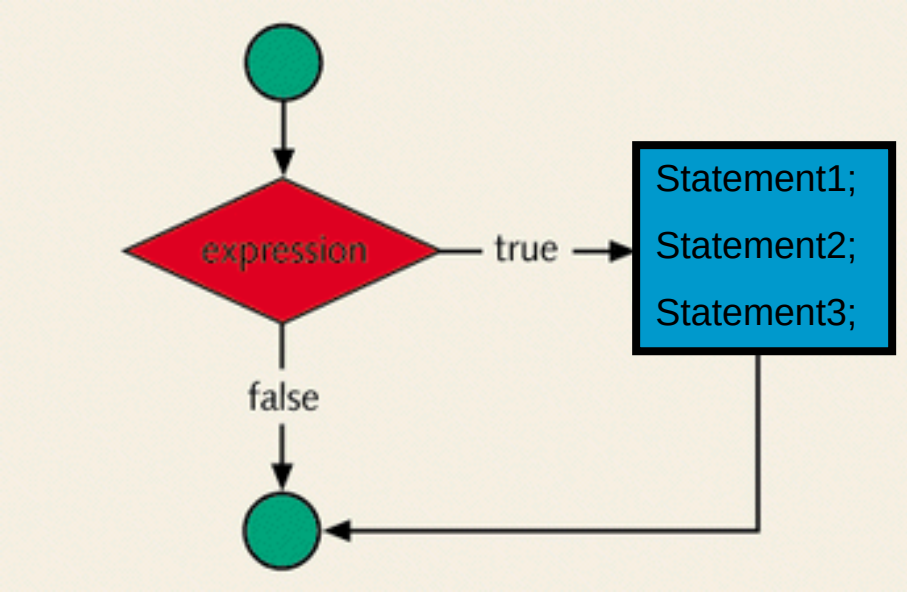
View sample program

- **Example**

```
if (x < 5) cout << "low x";
     else    cout << "high x";
```

Compound Statements

- Consider the need for <u>multiple</u> statements to be controlled by the `if`
- This is called a <u>compound</u> statement
- Group the statements in curly brackets



Compound Statements

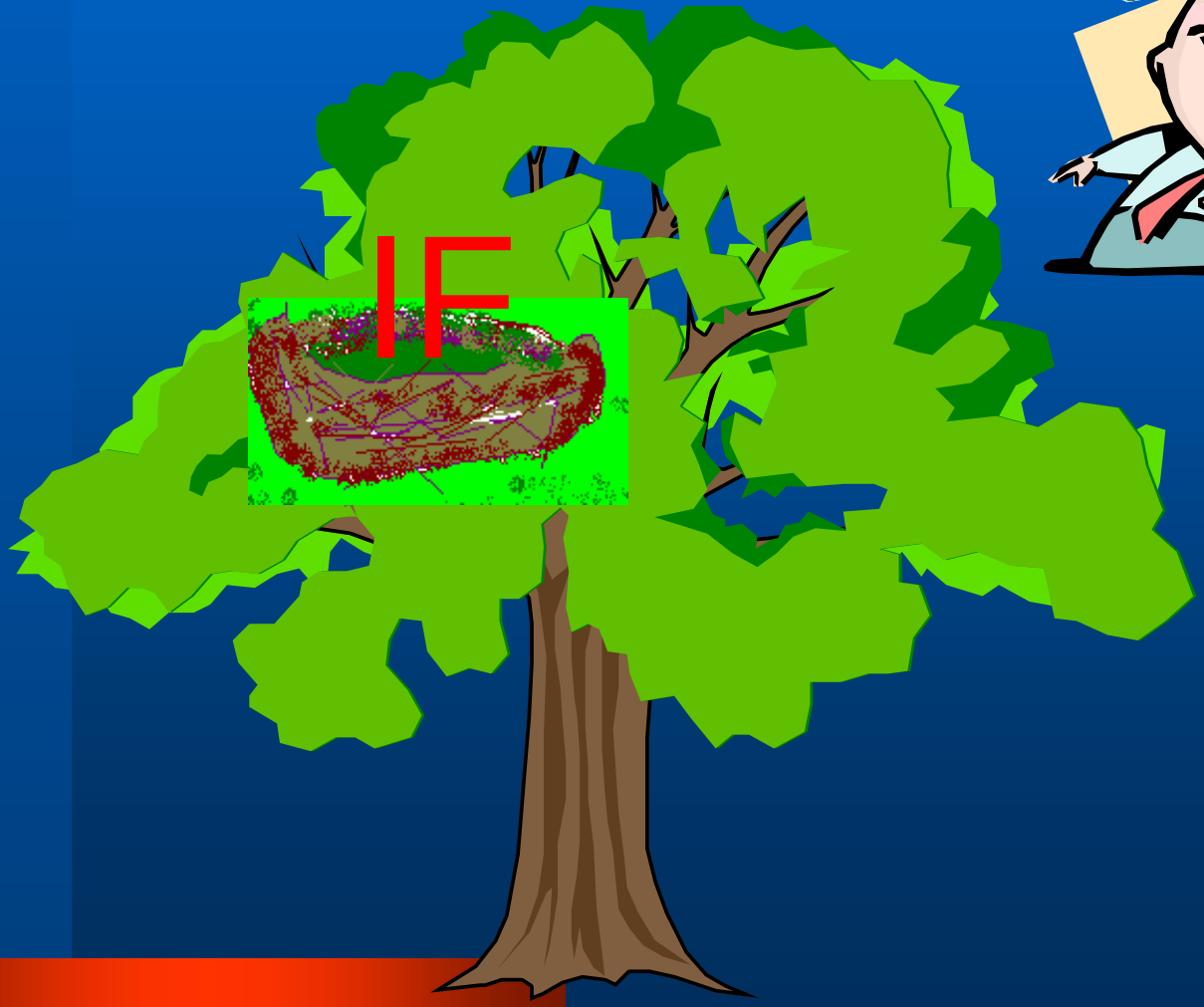
- **Example**

```
if (x < 5)
    {
        x = x + 10;
        cout << x;
    }
```

The compound statement

- **Note the use of indenting and white space in the source code for readability.**

The Nested `if`



Nested `if`

- Syntax calls for a "statement" after the if (…)
- That statement can be any kind of statement
  - (List statements we know about)
- It can be an if statement

cout
cin
assignment
if

```
if (x < 7)
  if (y > 5)
   cout << "hi mom";
```

The Dangling `else`

- **How to determine which `if` the `else` goes with**
- **Example:**

```
if (abs (x - 7))
  if (x < 7) cout << "x approaches 7 from left";
   ?   else
          cout << "x approaches 7 from the right";
  ?  else
          cout << "x not close to 7";
```

Rule : An ***else*** goes with the closest ***unmatched if***

The Dangling Else

- **Rule : an `else` goes with the closest unmatched `if`**

```
if (x < y)
   if (y > 3) cout << "message about y > 3";
        else  cout << "message about x and y";
```

- **Consider … how do you force an `else` to go with a previous `if`?**

Use { curly brackets } to nest the statements

```
if (x < y)
 { if (y > 3) cout << "message about y > 3"; }
        else  cout << "message about x and y";
```

Multiple Selections

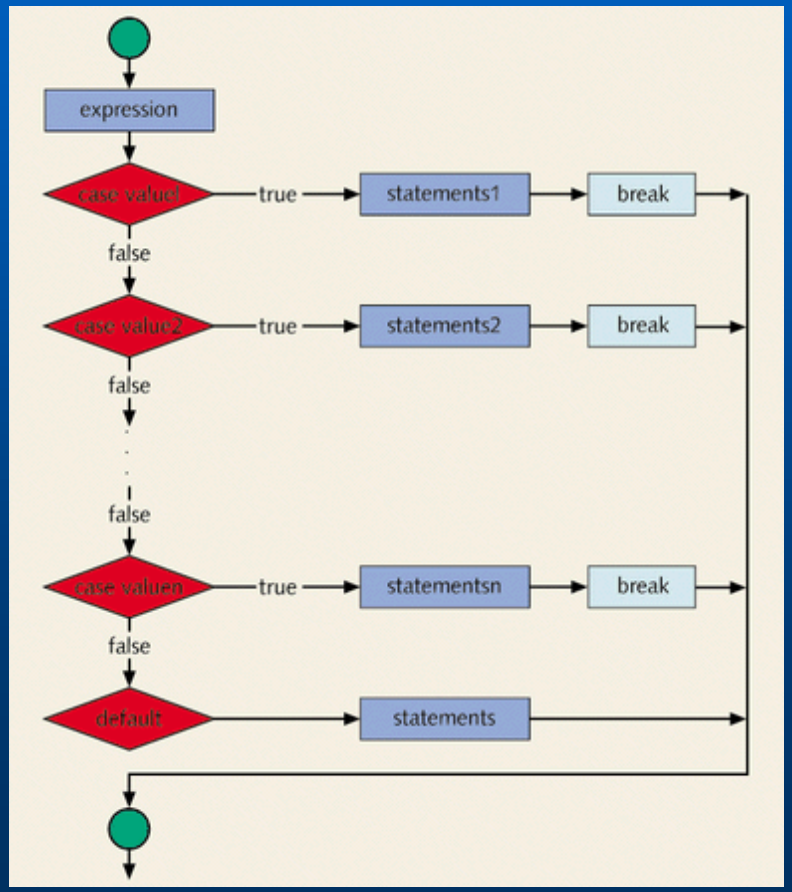
- **Consider determining a letter grade based on a score**
  - **Cut off points for A, B, C, and D are 90, 80, 70, and 60 respectively**
- **We check the score against each of these values**
- **See source code**

Multiple Selections

- **Contrast**
  - **A sequence of `if … else if …` statements**
  - **A sequence of separate `if` statements**
- **What happens in each case when it is the first if condition that is true?**
  - **`if … else if` sequence will jump out of the structure whenever match is found**
  - **sequence of separate `if`'s – each `if` is checked, no mater where the match is**

Multiple Selections

- Recall the current branching capability provided by the `if ( … )` statement
- Only branches two ways
- We desire a more eloquent way to do multiway branching



Switch Statement in C++

switch Structures

- Switch expression => the expression in parentheses whose value determines which switch label is selected
  - cannot be floating point
  - usually is `int` or `char`
- Identifiers following `case` must be constants

```
switch (choice) {
case 1 : do_option_one(); break;
case 2 :
case 3 : do_2_3_a ();
         do_2_3_b (); break;
default : do_something_else ();  }
```

Switch Statement in C++

- **Syntax**

```
switch (selector)
{ case L1: statements1; break;
  case L2: statements2; break;
   …
  default: statements_n;
 }
```

- **Semantics:**

This statement has the same meaning as in the algorithmic language.

Example 4: C++ Program

```
// File: music.cpp
/*The program corresponds to Example 5 on slide 31. It reads a letter and displays a suitable
    musical note (do, re, mi, etc.)*/
#include <iostream.h>
void main ( )
{ char ch ;
  cin >> ch ;
   switch  ( ch )
   { case ‘c’ : cout <<  “ do “ << endl; break;
     case ‘d’ : cout <<  “ re “ << endl; break;
     case ‘e’ : cout <<  “ mi “ << endl; break;
     case ‘f’ : cout <<  “ f “ << endl; break;
     case ‘g’ : cout <<  “ sol “ << endl; break;
     case ‘a’ : cout <<  “ la “ << endl; break;
     case ‘b’ : cout <<  “ ti “ << endl; break;
     default :  cout << “ Invalid note was read “ << endl;
}
```

Example 6: C++ Program

```
// File: circle.cpp
/*This program corresponds to Example 6 on slide 33. It is a menu driven
program. It calculates the area of the circle if letter  a  is read or calculates the
circumference if letter  c  is read.*/
#include <iostream.h>
void main ( )
{ char ch ;    float radius, area, circum;
   cout << " Enter the radius of a circle: " ;
   cin >>   radius;
   cout << " Enter a to calculate the area of a circle or c to calculate its
circumference:"
   cin >> ch ;
   switch (ch)
    { case 'a' :  area = 3.14f * radius * radius;
                  cout <<  " Area = " << area << endl;  break;
      case'c' :   circum = 2 * radius * 3.14f ;
                  cout << " Circumference =  " << circum << endl; break;
      default :   cout << " Invalid letter was read " << endl;
    }
}
```

```
#include <iostream.h>
void main ( )
{ char ch ;
   cout << " \n Enter the grade of student: "<<endl ;
   cin >>   ch;
 switch (ch)
{
case  'A' :
case   'a' :
            cout<<"Excellent";
             break;
case  'B' :
case   'b' :
            cout<<"Good";
            break;
case  'C' :
case   'c' :
            cout<<"O.K";
            break;
case  'D' :
case   'd' :
case  'F' :
case   'f' :
            cout<<"poor";
            break;
default:    cout<<"invalid letter grade";
}
}
```

```
#include <iostream.h>
void main()
{ int x,y;
 cout << "Enter 2 integer number: ";
 cin >> x>>y;
 switch (x+y)
 {  case 7: cout << "Too small, sorry!";
                break;
   case 5: cout << "Good job!\n";
                break;
   case 4: cout << "Nice Pick!\n";
   case 3: cout << "Excellent!\n";
               break;
   case 2: cout << "Masterful!\n";
              break;
   case 1: cout << "Incredible!\n";
                break;
   default: cout << "Too large!\n";
 }
cout << "\n\n";
 }
```